

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 2 – Schema Analysis and Viva Voce
Weightage:		Total Marks:	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	P.Sunayana	Reg. No.:	192424303
Section / Batch:	Slot-A 2024	Date of Submission:	03-08-2026
Faculty In charge:	K. Veena Devi	Project Mode:	Individual Submission

Code of Conduct

I _____P.Sunayana 192424393_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____P.Sunayana_____

Identification of Entities and Attributes	Understanding of Relationships (4 Marks)	Analysis of Keys and Constraints (4 Marks)	Detection of Design Issues (4 Marks)	Viva Voce (4 Marks)	Total (20 Marks)
--	---	---	---	----------------------------	-------------------------

(4 Marks)					

1. University Course Registration System

A university database contains the following tables:

- Student(StudentID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Course(CourseID, CourseName, FacultyID)
- Faculty(FacultyID, FacultyName)
- Enrollment(StudentID, CourseID, Semester)

A student can enroll in multiple courses, and each course can have multiple students.

Entities and Attributes

- **Student** (StudentID, Name, DepartmentID)
- **Department** (DepartmentID, DepartmentName)
- **Course** (CourseID, CourseName, FacultyID)
- **Faculty** (FacultyID, FacultyName)
- **Enrollment** (StudentID, CourseID, Semester)

Relationships

- Department → Student (**1:M**)
- Faculty → Course (**1:M**)
- Student ↔ Course (**M:N**) through Enrollment

Keys and Constraints

- Primary Keys: StudentID, DepartmentID, CourseID, FacultyID
- Enrollment: Composite PK (StudentID, CourseID, Semester)
- Foreign Keys:

- DepartmentID → Department
- FacultyID → Faculty
- StudentID → Student
- CourseID → Course

Design Issues

- Enrollment avoids duplicate course information.
- Schema is normalized and minimizes redundancy.

Viva Questions

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

It handles the many-to-many relationship between students and courses.

A student can take many courses and each course has many students.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

Add a Grade attribute to the Enrollment table.

2. Hospital Management System

A hospital database contains:

- Patient(PatientID, Name, Age)
- Doctor(DoctorID, DoctorName, Specialization)
- Appointment(AppointmentID, PatientID, DoctorID, Date)
- Prescription(PrescriptionID, AppointmentID, Medicine)

Multiple patients can consult the same doctor.

Entities

- Patient
- Doctor

- Appointment
- Prescription

Relationships

- Patient → Appointment (**1:M**)
- Doctor → Appointment (**1:M**)
- Appointment → Prescription (**1:M**)

Keys

- Primary Keys: PatientID, DoctorID, AppointmentID, PrescriptionID
- Foreign Keys:
 - PatientID → Patient
 - DoctorID → Doctor
 - AppointmentID → Appointment

Design Issues

- Separate prescription table avoids repeating medicine information.

Viva Questions

Q1. What problems might occur if prescription details are stored directly in the Appointment table?

Multiple medicines cannot be stored efficiently.

Causes redundancy and update problems.

Q2. How would the schema change if a prescription contains multiple medicines?

Keep Prescription table with one record for each medicine.

3. Online Shopping System

Database tables include:

- Customer(CustomerID, Name)
- Product(ProductID, ProductName, Price)
- Order(OrderID, CustomerID, OrderDate)
- OrderDetails(OrderID, ProductID, Quantity)

Each order may contain multiple products.

Entities

- Customer
- Product
- Order
- OrderDetails

Relationships

- Customer → Order (**1:M**)
- Order ↔ Product (**M:N**) through OrderDetails

Keys

- Primary Keys:
 - CustomerID
 - ProductID
 - OrderID
- OrderDetails:
 - Composite PK (OrderID, ProductID)

Design Issues

- OrderDetails stores quantity and multiple products.

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

One order can contain many products.

Stores quantity of each product.

Q2. What would happen if ProductID were stored directly in the Order table?

Only one product per order could be stored.

Not suitable for real shopping systems.

4. Library Management System

Database tables:

- Member(MemberID, Name)
- Book(BookID, Title, Author)
- Issue(MemberID, BookID, IssueDate, ReturnDate)

Members can borrow multiple books.

Entities

- Member
- Book
- Issue

Relationships

- Member ↔ Book (**M:N**) through Issue

Keys

- Primary Keys:
 - MemberID
 - BookID
- Issue:
 - Composite PK (MemberID, BookID, IssueDate)

Viva Questions

Q1. Explain the relationship between Member and Book through the Issue table.

Issue connects members and books.

A member can borrow many books.

A book can be borrowed multiple times over time.

Q2. How would you modify the schema to track overdue fines? \

Add FineAmount and FineStatus to Issue.

5. Banking System

Tables:

- Customer(CustomerID, Name)
- Account(AccountNo, CustomerID, Balance)
- Transaction(TransactionID, AccountNo, Amount, Type)

One customer can have multiple accounts.

Entities

- Customer
- Account
- Transaction

Relationships

- Customer → Account (**1:M**)
- Account → Transaction (**1:M**)

Keys

- CustomerID
- AccountNo
- TransactionID

Design Issues

- Separate transactions maintain account history.

Viva Questions

Q1. Why should transactions be stored separately from account details?

Keeps complete transaction history.

Prevents overwriting old records.

Q2. How would you redesign the schema to support joint accounts?

Create CustomerAccount(CustomerID, AccountNo) bridge table.

6. Employee Payroll System

Tables:

- Employee(EmployeeID, Name, DepartmentID)
- Department(DepartmentID, DepartmentName)
- Payroll(PayrollID, EmployeeID, Salary, Month)

Employees belong to departments and receive monthly salaries.

Entities

- Employee
- Department
- Payroll

Relationships

- Department → Employee (**1:M**)
- Employee → Payroll (**1:M**)

Keys

- EmployeeID
- DepartmentID
- PayrollID

Design Issues

- Department data stored once.

Viva Questions

Q1. Why is Department information separated into a different table?

Avoids duplicate department information.

Easier updates.

Q2. What redundancy issues would arise if department details were stored in Employee records?

Department name repeated for every employee.

Update anomalies occur.

7. Hotel Reservation System

Tables:

- Customer(CustomerID, Name)
- Room(RoomID, RoomType)
- Reservation(ReservationID, CustomerID, RoomID, CheckIn, CheckOut)

Rooms can be booked by different customers at different times.

Entities

- Customer
- Room
- Reservation

Relationships

- Customer → Reservation (**1:M**)
- Room → Reservation (**1:M**)

Keys

- CustomerID
- RoomID
- ReservationID

Design Issues

- Reservation stores booking dates separately.

Viva Questions

Q1. Why should reservation details be maintained separately from room details?

Tracks booking history.

Rooms can have many bookings over time.

Q2. How would you prevent double booking of rooms using the schema?

Check for overlapping CheckIn and CheckOut dates before inserting a reservation.

8. Food Delivery Application

Tables:

- Customer(CustomerID, Name)

- Restaurant(RestaurantID, Name)
- Order(OrderID, CustomerID, RestaurantID)
- DeliveryAgent(AgentID, Name)
- Delivery(OrderID, AgentID)

Orders are delivered by delivery agents.

Entities

- Customer
- Restaurant
- Order
- DeliveryAgent
- Delivery

Relationships

- Customer $\rightarrow$ Order (**1:M**)
- Restaurant $\rightarrow$ Order (**1:M**)
- DeliveryAgent $\rightarrow$ Delivery (**1:M**)
- Order $\rightarrow$ Delivery (**1:1**)

Keys

- CustomerID
- RestaurantID
- OrderID
- AgentID

Design Issues

- Delivery table separates delivery assignment.

Viva Questions

Q1. Why is Delivery maintained as a separate table?

Allows assignment of delivery agents independently.

Supports delivery tracking.

Q2. How would the schema change if multiple delivery agents could handle a single order?

Delivery becomes a bridge table with multiple records for one OrderID.

9. Learning Management System (LMS)

Tables:

- Student(StudentID, Name)
- Course(CourseID, CourseName)
- Enrollment(StudentID, CourseID)
- Assignment(AssignmentID, CourseID)

Students can enroll in many courses.

Entities

- Student
- Course
- Enrollment
- Assignment

Relationships

- Student $\leftrightarrow$ Course (**M:N**) through Enrollment
- Course $\rightarrow$ Assignment (**1:M**)

Keys

- StudentID
- CourseID
- AssignmentID

Design Issues

- Enrollment removes redundancy.

Viva Questions

Q1. Why is Enrollment considered a bridge table?

Connects students and courses in a many-to-many relationship.

Q2. How would you modify the schema to record assignment grades?

Create Submission(StudentID, AssignmentID, Grade) table or add Grade to a Submission table.

10. Bus Ticket Reservation System

Tables:

- Passenger(PassengerID, Name)
- Bus(BusID, Route)
- Booking(BookingID, PassengerID, BusID, TravelDate)

Passengers can make multiple bookings.

Entities

- Passenger
- Bus
- Booking

Relationships

- Passenger → Booking (1:M)
- Bus → Booking (1:M)

Keys

- PassengerID
- BusID
- BookingID

Design Issues

- Booking stores travel details separately.

Viva Questions

Q1. Why should booking details be stored separately from passenger information?

A passenger can book multiple trips.

Stores travel history and booking information.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

Create BookingPassenger(BookingID, PassengerID) bridge table to support group bookings.